SAVEETHA SCHOOL OF ENGINEERING

C PROGRAMMING ASSIGNMENT

Comprehensive System Design and Implementation Challenge

Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Comprehensive System Design and Implementation Challenge – District Blood Bank Network
GitHub Repository URL	https://github.com/bavadarini28 / CSA0209-1912260562-BAVADARANI.S.git
Submission Date	13 NOVEMBER 2026

TASK B3 — Text File vs Binary File (8 Marks)

Task / Problem Statement

Write and read donor records in a text file and in a binary file, compare their correctness, size and speed at 1,000, 10,000 and 100,000 records, measure a single-record update, and recommend a format.

Aim

To measure the engineering differences between text and binary storage and to complete the in-place update function.

Concept / Theory

A text record is written with fprintf and read with fgets + parsing: numbers become characters and lines have variable length. A binary record is a copy of the struct's bytes written with fwrite: all records have the same size (sizeof(Donor) = 72 bytes here), so record *k* is at offset 72·*k* and can be read or overwritten with fseek. Binary files are fast but depend on the computer's byte order, int size and struct padding.

Procedure / Algorithm

1. Generate donor records deterministically (id, name, group, age, phone, last-donation date).
2. Write 10 records to a text and a binary file, read them back and compare every field.
3. For n = 1,000, 10,000, 100,000: write and read both files five times, keep the best time, and record the file size with stat.
4. Update the 50,000th record in place (binary: fseek, fread, fwrite) and by rewriting the whole text file (read all lines, change one, write to a temporary file, rename).
5. Complete TODO 1 of the given stock program and run it.
6. Compare the stock written one record at a time and with one fwrite for the whole array.

Program / Source Code

Program 1 – the given stock-update program with TODO 1 completed (b3_todo.c):

```
#include <stdio.h>

typedef struct { int group_code; int units; } Stock; /* fixed-size binary record */

/* Overwrites the units of record number index directly in the binary file. */
int update_units(FILE *fp, int index, int units)
{
    Stock s;
    if (fseek(fp, (long)index * (long)sizeof s, SEEK_SET) != 0) return 0; /* go to record */
    if (fread(&s, sizeof s, 1, fp) != 1) return 0;                  /* read it      */
    s.units = units;                                                /* change it    */
    if (fseek(fp, -(long)sizeof s, SEEK_CUR) != 0) return 0;        /* step back    */
    if (fwrite(&s, sizeof s, 1, fp) != 1) return 0;                 /* write it     */
    return 1;
}

int main(void)
{
    FILE *fp = tmpfile();
    if (fp == NULL) { perror("tmpfile"); return 1; }
    Stock initial[5] = {{1, 40}, {2, 25}, {3, 12}, {4, 8}, {5, 30}};
    fwrite(initial, sizeof(Stock), 5, fp);

    printf("update record 3: %s\n", update_units(fp, 3, 20) ? "done" : "failed");
    printf("update record 9: %s\n", update_units(fp, 9, 20) ? "done" : "failed");

    rewind(fp);
    Stock s;
    while (fread(&s, sizeof s, 1, fp) == 1) printf("%d:%d ", s.group_code, s.units);
    printf("\n");
```

```
    fclose(fp);
    return 0;
}
```

Program 2 – comparison and timing program (b3_files.c), compiled with gcc -O2 -Wall -o b3_files b3_files.c:

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <sys/stat.h>

typedef struct { int id; char name[32]; char group[4]; int age; char phone[16]; char last[11]; } Donor;
typedef struct { int group_code; int units; } Stock;

static const char *G[8] = {"A+","A-","B+","B-","AB+","AB-","O+","O-"};
static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); return t.tv_sec + t.tv_nsec /
1e9; }
static long fsize(const char *p) { struct stat s; return stat(p, &s) ? -1 : (long)s.st_size; }

static void make(Donor *d, int i)
{
    memset(d, 0, sizeof *d);                    /* clears padding so binary compare is exact */
    d->id = 10000 + i;
    snprintf(d->name, sizeof d->name, "Donor%07d", i);
    strcpy(d->group, G[i % 8]);
    d->age = 18 + (i * 7) % 48;
    snprintf(d->phone, sizeof d->phone, "98%08d", i % 100000000);
    snprintf(d->last, sizeof d->last, "2026-%02d-%02d", 1 + i % 10, 1 + i % 28);
}
static void write_text(const char *p, int n)
{
    FILE *f = fopen(p, "w"); if (!f) { perror(p); exit(1); }
    Donor d;
    for (int i = 0; i < n; i++) { make(&d, i); fprintf(f, "%d,%s,%s,%d,%s,%s\n", d.id, d.name, d.group, d.age, d.phone,
d.last); }
    fclose(f);
}
static int read_text(const char *p, Donor *out, int max)
{
    FILE *f = fopen(p, "r"); if (!f) { perror(p); exit(1); }
    char line[256]; int n = 0;
    while (n < max && fgets(line, sizeof line, f)) {
        Donor *d = &out[n]; memset(d, 0, sizeof *d);
        if (sscanf(line, "%d,%31[^,],%3[^,],%d,%15[^,],%10s", &d->id, d->name, d->group, &d->age, d->phone, d-
>last) != 6) break;
        n++;
    }
    fclose(f); return n;
}
static void write_bin(const char *p, int n)
{
    FILE *f = fopen(p, "wb"); if (!f) { perror(p); exit(1); }
    Donor d;
    for (int i = 0; i < n; i++) { make(&d, i); fwrite(&d, sizeof d, 1, f); }
    fclose(f);
}
static int read_bin(const char *p, Donor *out, int max)
{
    FILE *f = fopen(p, "rb"); if (!f) { perror(p); exit(1); }
```

```
    int n = (int)fread(out, sizeof(Donor), max, f);
    fclose(f); return n;
}
static int same(const Donor *a, const Donor *b)
{
    return a->id == b->id && !strcmp(a->name, b->name) && !strcmp(a->group, b->group) &&
           a->age == b->age && !strcmp(a->phone, b->phone) && !strcmp(a->last, b->last);
}

int main(void)
{
        printf("sizeof(Donor) = %zu bytes, sizeof(Stock) = %zu bytes\n\n", sizeof(Donor), sizeof(Stock));

    /* ---- B3.1 correctness on 10 records ---- */
    puts("== B3.1 write 10 records, read back, compare ==");
    Donor orig[10], t[10], b[10];
    for (int i = 0; i < 10; i++) make(&orig[i], i);
    write_text("small.txt", 10); write_bin("small.bin", 10);
    int nt = read_text("small.txt", t, 10), nb = read_bin("small.bin", b, 10);
    int okt = nt == 10, okb = nb == 10;
    for (int i = 0; i < 10; i++) { okt &= same(&orig[i], &t[i]); okb &= same(&orig[i], &b[i]); }
    printf("text  : %d records read, identical to original: %s\n", nt, okt ? "YES" : "NO");
    printf("binary: %d records read, identical to original: %s\n", nb, okb ? "YES" : "NO");
    printf("sample line of text file : %d,%s,%s,%d,%s,%s\n", t[3].id, t[3].name, t[3].group, t[3].age, t[3].phone, t[3].last);
    printf("small.txt = %ld bytes, small.bin = %ld bytes\n\n", fsize("small.txt"), fsize("small.bin"));

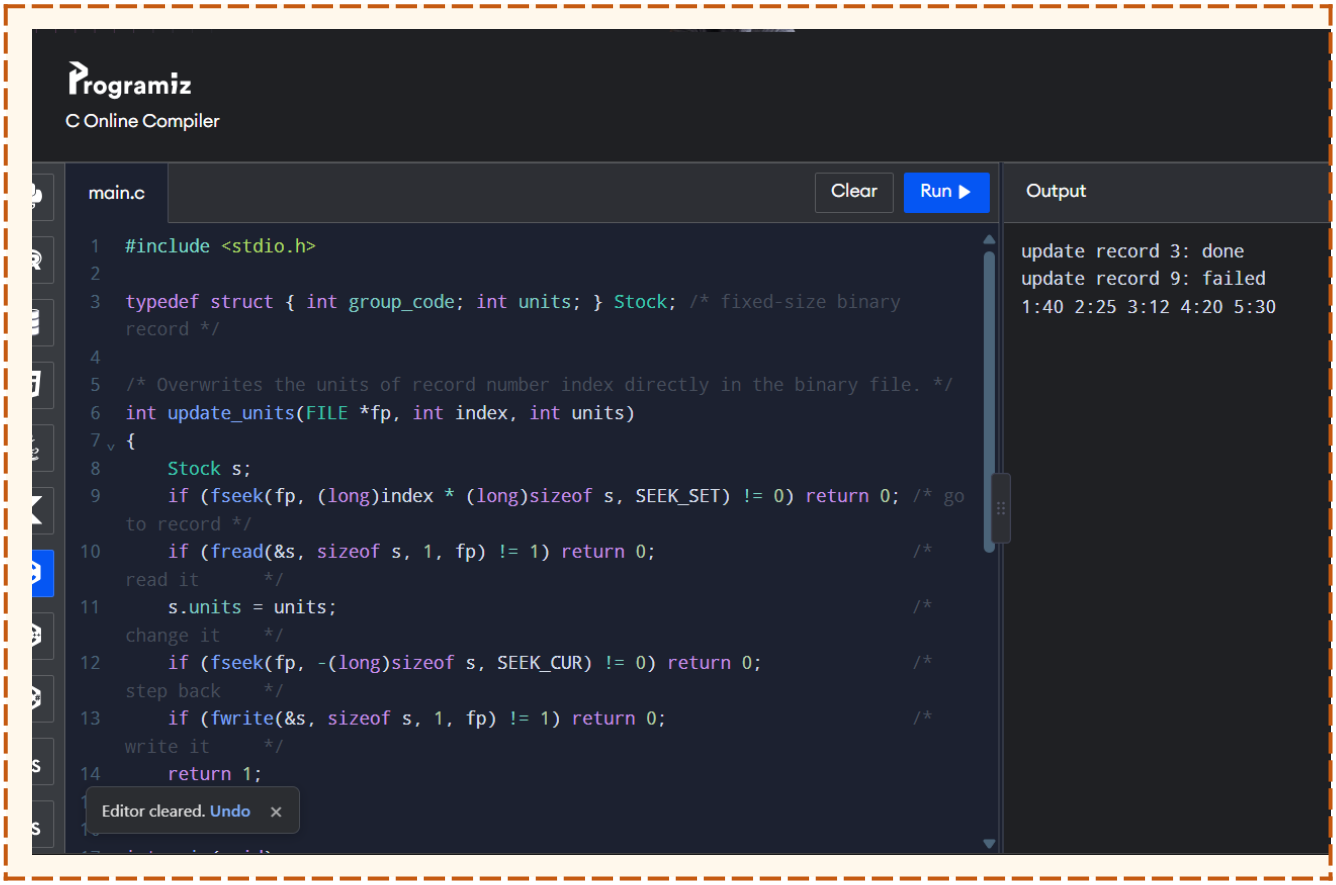
    /* ---- B3.2 sizes and times (best of 5) ---- */
    puts("== B3.2 file size and time (best of 5 runs, files in OS cache) ==");
    printf("%8s | %-6s | %12s | %10s | %10s\n", "records", "format", "size (bytes)", "write (ms)", "read (ms)");
    int sizes[] = {1000, 10000, 100000};
    for (int s = 0; s < 3; s++) {
        int n = sizes[s];
        Donor *buf = malloc((size_t)n * sizeof(Donor));
        double wt = 1e9, rt = 1e9, wb = 1e9, rb = 1e9;
        for (int r = 0; r < 5; r++) {
            double a = now(); write_text("d.txt", n); double e = now() - a; if (e < wt) wt = e;
        }
        for (int r = 0; r < 5; r++) { double a = now(); read_text("d.txt", buf, n); double e = now() - a; if (e < rt) rt = e; }
        for (int r = 0; r < 5; r++) { double a = now(); write_bin("d.bin", n); double e = now() - a; if (e < wb) wb = e; }
        for (int r = 0; r < 5; r++) { double a = now(); read_bin("d.bin", buf, n); double e = now() - a; if (e < rb) rb = e; }
        printf("%8d | %-6s | %12ld | %10.2f | %10.2f\n", n, "text", fsize("d.txt"), wt * 1e3, rt * 1e3);
        printf("%8d | %-6s | %12ld | %10.2f | %10.2f\n", n, "binary", fsize("d.bin"), wb * 1e3, rb * 1e3);
        free(buf);
    }

    /* ---- B3.3 update the 50,000th record ---- */
    puts("\n== B3.3 update record 50,000 (100,000 records on disk) ==");
    write_text("big.txt", 100000); write_bin("big.bin", 100000);
    long idx = 49999;                           /* 50,000th record, zero-based index */
    int reps = 200;
    double a = now();
    for (int r = 0; r < reps; r++) {
        FILE *f = fopen("big.bin", "r+b"); Donor d;
        fseek(f, idx * (long)sizeof d, SEEK_SET); fread(&d, sizeof d, 1, f);
        d.age = 30 + r % 10;
        fseek(f, -(long)sizeof d, SEEK_CUR); fwrite(&d, sizeof d, 1, f); fclose(f);
    }
    double tb = (now() - a) / reps;
    reps = 20; a = now();
    for (int r = 0; r < reps; r++) {
        FILE *in = fopen("big.txt", "r"), *out = fopen("big.tmp", "w"); char line[256]; long ln = 0;
        while (fgets(line, sizeof line, in)) {
```

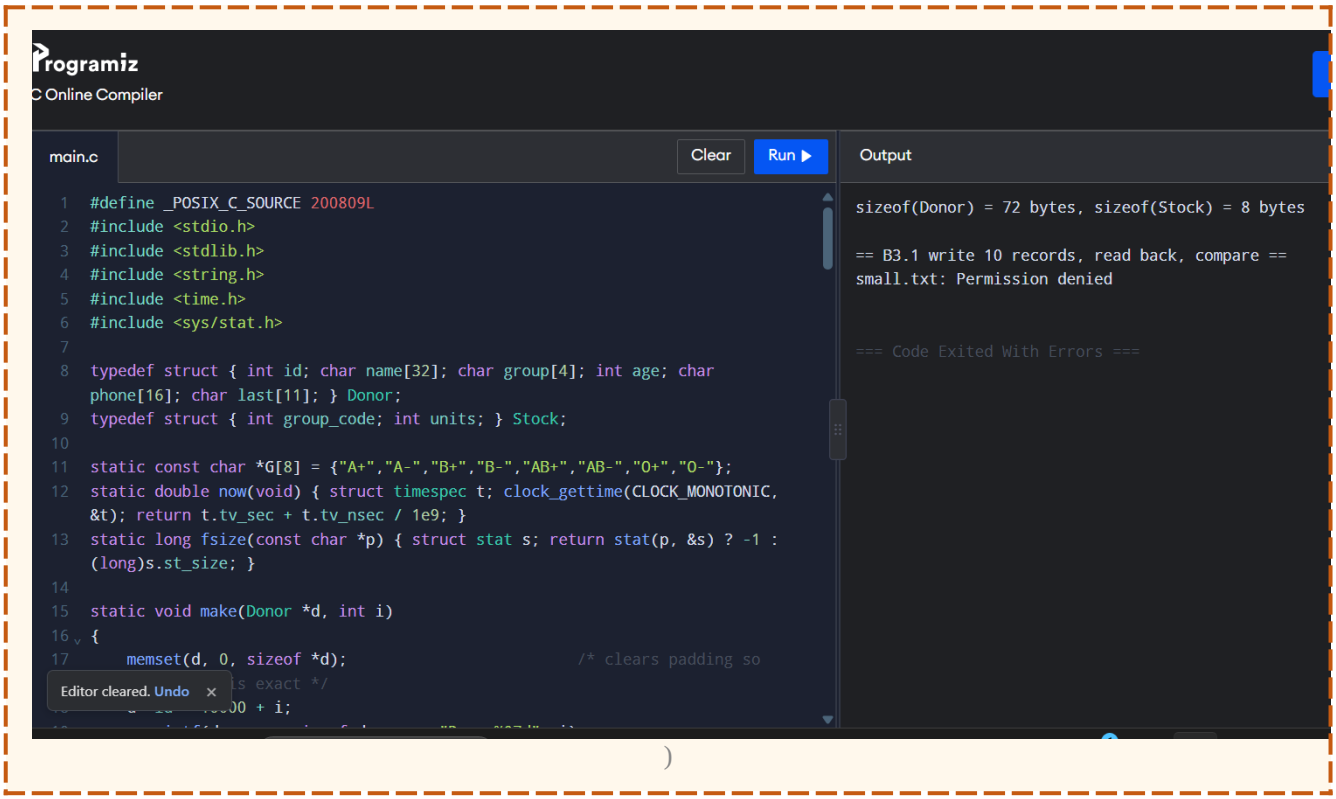
```
            if (ln == idx) { Donor d; sscanf(line, "%d,%31[^,],%3[^,],%d,%15[^,],%10s", &d.id, d.name, d.group, &d.age,
d.phone, d.last);
                d.age = 30 + r % 10; fprintf(out, "%d,%s,%s,%d,%s,%s\n", d.id, d.name, d.group, d.age, d.phone, d.last); }
            else fputs(line, out);
            ln++;
        }
        fclose(in); fclose(out); rename("big.tmp", "big.txt");
    }
    double tt = (now() - a) / reps;
    printf("binary, fseek in place : %10.4f ms per update (average of 200)\n", tb * 1e3);
    printf("text, rewrite whole file: %9.4f ms per update (average of 20)\n", tt * 1e3);
    printf("ratio text/binary       : %.0f times slower\n", tt / tb);

    /* ---- alternative: one fwrite for the array vs one per record ---- */
    puts("\n== Alternative: per-record fwrite vs one fwrite (Stock records) ==");
    printf("%10s | %14s | %14s | %12s\n", "records", "per-record ms", "one-call ms", "array bytes");
    int sn[] = {1000, 100000, 5000000};
    for (int s = 0; s < 3; s++) {
        int n = sn[s]; Stock *arr = malloc((size_t)n * sizeof(Stock));
        for (int i = 0; i < n; i++) { arr[i].group_code = i % 8; arr[i].units = i % 50; }
        double best1 = 1e9, best2 = 1e9;
        for (int r = 0; r < 3; r++) {
            FILE *f = fopen("st1.bin", "wb"); double a1 = now();
            for (int i = 0; i < n; i++) fwrite(&arr[i], sizeof(Stock), 1, f);
            fclose(f); double e = now() - a1; if (e < best1) best1 = e;
            f = fopen("st2.bin", "wb"); a1 = now(); fwrite(arr, sizeof(Stock), n, f); fclose(f);
            e = now() - a1; if (e < best2) best2 = e;
        }
        printf("%10d | %14.3f | %14.3f | %12zu\n", n, best1 * 1e3, best2 * 1e3, (size_t)n * sizeof(Stock));
        free(arr);
    }
    return 0;
}
```

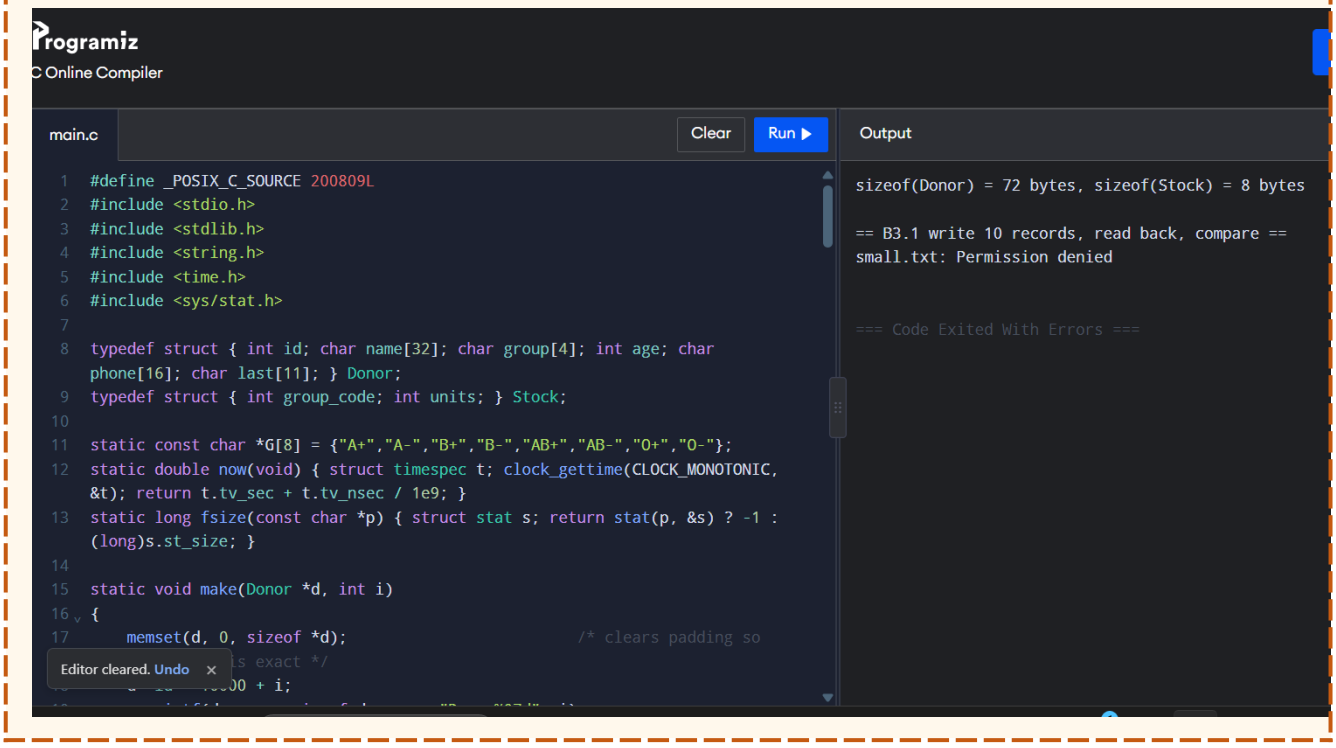
Output Screenshot




```
update record 3: done
update record 9: failed
1:40 2:25 3:12 4:20 5:30
```



)



Complete output of my run:

```
sizeof(Donor) = 72 bytes, sizeof(Stock) = 8 bytes

== B3.1 write 10 records, read back, compare ==
text  : 10 records read, identical to original: YES
binary: 10 records read, identical to original: YES
```

```
sample line of text file : 10003,Donor0000003,B-,39,9800000003,2026-04-04
small.txt = 472 bytes, small.bin = 720 bytes

== B3.2 file size and time (best of 5 runs, files in OS cache) ==
 records | format | size (bytes) | write (ms) |  read (ms)
    1000 | text   |       47250 |       0.28 |       0.20
    1000 | binary |       72000 |       0.21 |       0.00
   10000 | text   |      472500 |       2.97 |       2.07
   10000 | binary |      720000 |       2.09 |       0.03
  100000 | text   |     4735000 |      28.85 |      21.08
  100000 | binary |     7200000 |      18.50 |       0.66

== B3.3 update record 50,000 (100,000 records on disk) ==
binary, fseek in place :     0.0022 ms per update (average of 200)
text, rewrite whole file:    8.1426 ms per update (average of 20)
ratio text/binary       : 3696 times slower

== Alternative: per-record fwrite vs one fwrite (Stock records) ==
   records |  per-record ms |    one-call ms |  array bytes
      1000 |         0.030 |         0.005 |         8000
    100000 |         1.190 |         0.166 |       800000
   5000000 |        59.362 |        10.818 |     40000000
```

Analysis / Discussion

Why record 9 fails and why the file is not damaged. The file has 5 records (40 bytes). Record 9 would start at byte 9 × 8 = 72. For an ordinary file fseek beyond the end **succeeds**, but fread then finds nothing and returns 0, so update_units returns 0 at that step, **before** any fwrite. Nothing is written, so the file keeps its 5 records and the final listing is unchanged. (If the read check were left out and the write were allowed, the file would be extended with a gap of zero bytes – that would damage it; the check order is what protects the file.)

B3.1 – Correctness (2 marks). 10 records were written and read back from both formats; all six fields of every record matched (text: 472 bytes, binary: 720 bytes). The binary struct is cleared with memset before filling, so the padding bytes are also identical.

B3.2 – Size and time (2 marks).

Records	Format	Size (bytes)	Write (ms)	Read (ms)
1,000	text	47,250	0.28	0.20
1,000	binary	72,000	0.21	0.00 (< 0.01)
10,000	text	472,500	2.97	2.07
10,000	binary	720,000	2.09	0.03
100,000	text	4,735,000	28.85	21.08
100,000	binary	7,200,000	18.50	0.66

Both formats grow linearly with n (10× the records → about 10× the size and time). In this experiment the **binary file is bigger** (72 bytes against 47 per record) because every record has fixed-width fields (32-byte name, 16-byte phone, padding) while text stores only the characters used; text would be larger for short numbers stored as long fixed-width fields, so size depends on the data. Binary writing is about 1.5× faster and binary reading about 30× faster, because text needs number-to-characters conversion and parsing for every line, whereas binary reads all records with one fread and no conversion. (Files were in the operating-system cache, so these figures measure CPU and library cost, not the disk.)

B3.3 – Update one record (2 marks). Updating the 50,000th of 100,000 records took **0.0022 ms** in place in the binary file (average of 200) and **8.14 ms** by rewriting the text file (average of 20): about **3,700 times** slower. The binary update touches 72 bytes regardless of file size (O(1)); the text rewrite reads and writes 4.7 MB (O(n)).

B3.4 – Engineering trade-off (2 marks).

Criterion	Text file	Binary file
Update one record	Rewrite the file – O(n)	fseek + fwrite – O(1)
Read / write speed	Slower (conversion and parsing)	Faster (raw bytes)
Size	Only the characters used; depends on the data	Fixed width per record; includes padding
Human readable, editable, easy to diff / e-mail	Yes	No (needs a program)
Portability between computers	Good – same file on any computer (only line endings and character set may differ)	**Poor** – a struct written by one program may not be read correctly elsewhere: byte order (little/big endian), int and long sizes, struct padding / alignment and compiler differ. Fix: write each field separately in a fixed byte order, or keep the same program and platform
Damage by a bad write	Readable but a damaged line is visible	A shifted or wrong record is silent

Recommendation: stock → binary fixed-size records (tiny file, updated after every donation/issue, needs O(1) in-place update, used by the same program on the same computer). **Donors → text** (variable-length names and phone numbers, must be portable and easy to repair, backed up and exchanged between the 25 hospitals; the whole file is loaded once into memory, so its slower read does not matter – B4 loads 5 million donors in under 2 seconds).

Alternative Concept and Complexity Analysis – one `fwrite` for the whole array

fwrite(stock, sizeof(Stock), n, fp); /* one call */ versus for (i = 0; i < n; i++) fwrite(&stock[i], sizeof(Stock), 1, fp); /* n calls */

Records n	Per-record: library calls	One fwrite: calls	Per-record time (ms)	One-call time (ms)	Array to hold in memory
1,000	1,000	1	0.030	0.005	8 KB
100,000	100,000	1	1.190	0.166	800 KB
5,000,000	5,000,000	1	59.4	10.8	40 MB

Both are O(n) in time because n records of data must be written. The single call removes n − 1 function calls and runs about 5–7 times faster, because the library copies one large block into its buffer. The price is that the whole array (n × 8 bytes) must already be in memory – trivial for 8 stock records, but 40 MB for 5 million. The per-record version needs only one record in memory and is the safer choice when the data are generated or read one at a time. (The operating system sees about the same number of disk writes in both, since stdio buffers the per-record calls.)

Result

Both formats reproduced the data exactly. Binary was faster to read and write and about 3,700 times faster for a single update, text was more compact in this data set and portable. The recommendation is binary for stock and text for donors; the given TODO was completed and the failed update of record 9 was explained.